PES UNIVERSITY, BENGALURU

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SOFTWARE ENGINEERING LAB – 5

STATIC CODE ANALYSIS

Name: Sahana K Golabanvi

SRN: PES1UG25AM811

Project: Hostel Room Allocation System

Tools Used: Pylint, Bandit, Flake8

1. Objective

To improve the code quality, security, and coding style of the Hostel Room Allocation System using static code analysis tools such as Pylint, Bandit, and Flake8.

The objective is to identify programming errors, security vulnerabilities, and formatting issues, correct them, and verify the improvements through before-and-after analysis reports.

2. Introduction

Static Code Analysis is a software testing technique used to examine source code without executing the program.

In this experiment, a Python-based Hostel Room Allocation System was analyzed using three tools:

1. **Pylint:** Checks code quality, naming conventions, unused imports, and programming practices.
2. **Bandit:** Identifies potential security vulnerabilities, including insecure functions and unsafe coding practices.
3. **Flake8:** Checks code formatting, indentation, line length, and Python coding style.

The issues identified by these tools were corrected to improve the quality, security, readability, and maintainability of the program.

3. Tools Used

Tool	Purpose
Pylint	Identifies code-quality issues, naming violations, unused imports, and missing docstrings.
Bandit	Detects security vulnerabilities such as unsafe YAML loading, hardcoded passwords, and insecure functions.
Flake8	Identifies formatting issues, indentation errors, extra blank lines, and line-length violations.
Visual Studio Code	Used to edit, execute, and debug the Python program.
Git and GitHub	Used for version control, commits, and submission.

4. Procedure

1. Forked the assigned Hostel Room Allocation System GitHub repository.
2. Cloned the repository into Visual Studio Code.
3. Created and activated a Python virtual environment.
4. Installed Pylint, Bandit, and Flake8.
5. Executed the original hostel_allocation.py program.
6. Ran all three static analysis tools and generated before-analysis reports.
7. Identified code-quality, security, and formatting issues.
8. Modified the Python program to correct the identified issues.
9. Reran Pylint, Bandit, and Flake8 to verify the corrections.
10. Saved the before-and-after analysis reports.
11. Created three Git commits and pushed the corrected code and reports to GitHub.

5. Issues Identified and Fixes Applied

Issue Identified	Type	Description	Fix Applied
Mutable default argument	Bug	preferred=[] could reuse the same list across function calls.	Replaced with preferred=None and initialized an empty list inside the function.
Incorrect room capacity check	Logical Bug	Rooms could be allocated beyond their capacity.	Changed > to >= in the capacity condition.
Duplicate student allocation	Logical Bug	The same student could be allocated multiple rooms.	Added a check for existing allocations.
Incorrect room vacation	Logical Bug	Vacated students remained in room occupancy records.	Removed the student from both the room occupants list and allocation records.
Unsafe YAML loading	Security	yaml.load() with yaml.Loader could process unsafe YAML content.	Replaced with yaml.safe_load().
Hardcoded password	Security	Database credentials were directly included in source code.	Used the HOSTEL_DB_PASSWORD environment variable.
Insecure remote logging	Security	logging.config.listen() could accept unsafe remote configuration.	Removed the unused remote logging listener.
Insecure random generator	Security	random.sample() was flagged for security-sensitive randomness.	Used secrets.SystemRandom() for the room lottery.
Naming and docstring violations	Code Quality	Functions used camelCase names and lacked documentation.	Renamed functions to snake_case and added docstrings.
Formatting violations	Style	Incorrect indentation, extra blank lines, and long lines violated Flake8 rules.	Corrected indentation, spacing, and line lengths.

6. Before-and-After Analysis Results

The Hostel Room Allocation System was analyzed using Pylint, Bandit, and Flake8 before and after correcting the identified issues.

Analysis Tool	Before Analysis	After Analysis
Pylint	5.84/10	10.00/10
Bandit	Security vulnerabilities detected	0 issues detected
Flake8	Formatting and style violations detected	0 errors

6.1 Pylint Results

The initial Pylint score was **5.84/10**. After correcting unused imports, mutable default arguments, naming conventions, missing docstrings, and code-quality issues, the final score improved to **10.00/10**.

6.2 Bandit Results

The original program contained potentially insecure coding practices, including unsafe YAML loading, hardcoded credentials, and insecure remote logging.

After correcting the identified security issues, Bandit reported **zero High, Medium, and Low severity findings**.

6.3 Flake8 Results

The initial analysis identified formatting violations, including incorrect indentation, excessive line lengths, and missing or extra blank lines.

After correcting these issues, the final Flake8 analysis generated an empty report, indicating **zero formatting violations**.

7. Result

The static code analysis experiment was completed successfully. The Python program was improved using Pylint, Bandit, and Flake8.

The final Pylint score was 10.00/10, Bandit identified no issues, and Flake8 reported zero violations. The corrected code and analysis reports were uploaded to GitHub.

8. Reflection Questions and Answers

Q1. What is static code analysis?

Static code analysis is the process of examining source code without executing it. It helps identify programming errors, security vulnerabilities, and coding style violations.

Q2. What is the difference between Pylint, Bandit, and Flake8?

Pylint checks code quality and programming practices. Bandit identifies potential security vulnerabilities. Flake8 checks formatting, indentation, and coding style.

Q3. How did static code analysis improve the Hostel Room Allocation System?

Static analysis helped identify unsafe coding practices, incorrect function naming, missing documentation, and security vulnerabilities. Fixing these issues improved the readability, security, and maintainability of the code.

Q4. Which security issues were identified and how were they corrected?

The program contained unsafe YAML loading, a hardcoded password, insecure remote logging, and a non-cryptographic random generator. These were addressed using yaml.safe_load(), environment variables, removal of the unused logging listener, and secrets.SystemRandom().

Q5. Why is it important to run static analysis tools again after fixing issues?

Rerunning static analysis tools verifies whether previously reported issues have been resolved and helps identify any new issues introduced during modification.

Q6. What did you learn from this experiment?

I learned how to use Pylint, Bandit, and Flake8 to identify and correct issues in Python programs. I also learned the importance of secure coding practices, documentation, consistent formatting, and version control.

9. Conclusion

The Static Code Analysis experiment was successfully completed using Pylint, Bandit, and Flake8 on the Hostel Room Allocation System.The Pylint score improved from **5.84/10 to 10.00/10**. The final Bandit report identified zero security issues, and the Flake8 report contained no formatting violations.The experiment demonstrated how static analysis tools help improve software quality, security, readability, and maintainability.

The corrected Python source code and before-and-after analysis reports were maintained using Git and uploaded to GitHub.